

Module 2

Unit Test Frameworks — unittest, PyTest & Page Object Model

Module 2 of 4 — Selenium / QA Automation Training
Detailed Curriculum Report with Explanations, Examples & Code

Raw Selenium scripts quickly become unmanageable: no structured assertions, no reporting, no reusable setup/teardown, and no way to run subsets of tests. Test frameworks solve this by giving your automation a disciplined structure, built-in assertions, fixtures, parameterization, and report generation.

2.1 unittest

unittest is Python's built-in xUnit-style testing framework (bundled with the standard library, no installation needed). Tests are written as methods inside a class that extends `unittest.TestCase`.

First Test Case

test_login_unittest.py

```
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestLogin(unittest.TestCase):

    def test_valid_login(self):
        driver = webdriver.Chrome()
        driver.get("https://www.saucedemo.com")
        driver.find_element(By.ID, "user-name").send_keys("standard_user")
        driver.find_element(By.ID, "password").send_keys("secret_sauce")
        driver.find_element(By.ID, "login-button").click()
        self.assertIn("inventory.html", driver.current_url)
        driver.quit()

if __name__ == "__main__":
    unittest.main()
```

Class-Level setUp and tearDown Methods

`setUp()` runs before every test method (great for opening the browser), and `tearDown()` runs after every test method (great for closing it) — this removes duplicated boilerplate across tests.

setup_teardown.py

```

import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By

class TestLogin(unittest.TestCase):

    def setUp(self):
        self.driver = webdriver.Chrome()
        self.driver.get("https://www.saucedemo.com")

    def test_valid_login(self):
        self.driver.find_element(By.ID, "user-name").send_keys("standard_user")
        self.driver.find_element(By.ID, "password").send_keys("secret_sauce")
        self.driver.find_element(By.ID, "login-button").click()
        self.assertIn("inventory.html", self.driver.current_url)

    def test_invalid_login(self):
        self.driver.find_element(By.ID, "user-name").send_keys("bad_user")
        self.driver.find_element(By.ID, "password").send_keys("wrong_pass")
        self.driver.find_element(By.ID, "login-button").click()
        error = self.driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
        self.assertIn("do not match", error)

    def tearDown(self):
        self.driver.quit()

```

Asserting a Test Method

assertions.py

```

self.assertEqual(actual, expected)
self.assertTrue(condition)
self.assertFalse(condition)
self.assertIn("inventory", driver.current_url)
self.assertIsNotNone(element)

```

Creating a Test Suite

test_suite.py

```

import unittest
from test_login import TestLogin
from test_checkout import TestCheckout

def suite():
    suite = unittest.TestSuite()
    suite.addTest(unittest.makeSuite(TestLogin))
    suite.addTest(unittest.makeSuite(TestCheckout))
    return suite

if __name__ == "__main__":
    runner = unittest.TextTestRunner(verbosity=2)
    runner.run(suite())

```

2.2 PyTest Framework

PyTest is a third-party Python testing framework, and the industry standard for Selenium automation in Python. It requires far less boilerplate than unittest (plain `assert` statements, no need to subclass anything), and offers powerful fixtures, parameterization, plugins, and reporting.

Installing PyTest

```
pip install pytest
pip install pytest-html # for HTML reports
```

Naming Conventions

- Test files must be named `test_*.py` or `*_test.py`.
- Test functions must start with `test_`.
- Test classes must start with `Test` and contain no `__init__` method.

Test Files, Test Methods & Assertions

test_login.py

```
# test_login.py
def test_valid_login(driver):
    driver.get("https://www.saucedemo.com")
    driver.find_element("id", "user-name").send_keys("standard_user")
    driver.find_element("id", "password").send_keys("secret_sauce")
    driver.find_element("id", "login-button").click()
    assert "inventory.html" in driver.current_url
```

Subsets and Test Suites (Markers)

markers.py

```
import pytest

@pytest.mark.smoke
def test_login_page_loads(driver):
    driver.get("https://www.saucedemo.com")
    assert driver.title == "Swag Labs"

@pytest.mark.regression
def test_full_checkout_flow(driver):
    ...

# Run only smoke tests from the terminal:
# pytest -m smoke
```

Register custom markers in `pytest.ini` to avoid warnings:

pytest.ini

```
# pytest.ini
[pytest]
markers =
    smoke: quick sanity checks
    regression: full regression suite
```

Fixtures and conftest.py

Fixtures provide reusable setup/teardown logic that PyTest injects into test functions as arguments. `conftest.py` is a special file PyTest auto-discovers, letting fixtures be shared across every test file in the directory without importing them.

conftest.py

```
# conftest.py
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    drv = webdriver.Chrome()
    drv.implicitly_wait(5)
    yield drv          # value handed to the test
    drv.quit()         # teardown - runs after the test finishes
```

PyTest Features

- Plain `assert` statements with automatic, detailed failure introspection.
- Fixtures with scope control (function, class, module, session).
- Parameterization with `@pytest.mark.parametrize`.
- Rich plugin ecosystem: `pytest-html`, `pytest-xdist` (parallel run), `pytest-rerunfailures`, `allure-pytest`.
- Powerful command-line filtering (`-k`, `-m`, `-v`).

Running Multiple Test Cases & Parameterization

test_parametrize.py

```
import pytest

@pytest.mark.parametrize("username,password,expected_error", [
    ("locked_out_user", "secret_sauce", "Sorry, this user has been locked out."),
    ("standard_user", "wrong_pass", "do not match"),
    ("", "", "Username is required"),
])
def test_login_negative(driver, username, password, expected_error):
    driver.get("https://www.saucedemo.com")
    driver.find_element("id", "user-name").send_keys(username)
    driver.find_element("id", "password").send_keys(password)
    driver.find_element("id", "login-button").click()
    error = driver.find_element("css selector", "h3[data-test='error']").text
    assert expected_error in error
```

CLI usage

```
pytest test_login.py -v          # run one file, verbose
pytest -k login                  # run tests matching keyword 'login'
pytest -m smoke --maxfail=1
```

Moving Selenium Tests to PyTest & HTML Reports

Generate an HTML report

```
pip install pytest-html
pytest --html=report.html --self-contained-html
```

The generated report .html shows pass/fail counts, durations, and (with a small hook) can embed screenshots for failed tests directly in the report.

conftest.py (screenshot-on-failure hook)

```
# conftest.py - attach a screenshot to the PyTest HTML report on failure
import pytest

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver:
            driver.save_screenshot(f"{item.name}_failure.png")
```

2.3 Page Object Model (POM)

Why a Framework?

As a test suite grows, locators scattered across dozens of test methods become a maintenance nightmare: one small UI change breaks the same locator in ten different files. A framework enforces separation of concerns so a single change is made in exactly one place.

What Is the Page Object Model?

POM is a design pattern where each web page (or major component) of the application under test is represented by a corresponding class. The class stores that page's locators and exposes methods representing user actions on that page (e.g. `login()`). Test scripts then simply call these methods — they contain no locators and no low-level Selenium calls at all.

Base Class, Page Class, Test Class, Utility Class, Config File, Screenshot Util

Layer	Responsibility
BasePage	Common reusable methods every page needs: <code>click()</code> , <code>type()</code> , <code>wait_for()</code> , <code>get_text()</code> etc.
Page classes (LoginPage, InventoryPage)	Locators + page-specific action methods. Inherit from BasePage.
Test classes	Only test logic and assertions; call page-object methods, never raw Selenium calls.
Utility classes	Helpers: read Excel/CSV/JSON, generate random data, date utilities.
Configuration file	Base URL, browser choice, timeouts, environment (QA/Staging/Prod) - kept out of test code
Screenshot utility	Central <code>take_screenshot()</code> called from <code>conftest.py</code> hooks on failure.

BasePage and Util Concept / Inheriting BasePage

pages/base_page.py

```
# pages/base_page.py
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(driver, 10)

    def click(self, locator):
        self.wait.until(EC.element_to_be_clickable(locator)).click()

    def type(self, locator, text):
        elem = self.wait.until(EC.visibility_of_element_located(locator))
        elem.clear()
        elem.send_keys(text)

    def get_text(self, locator):
        return self.wait.until(EC.visibility_of_element_located(locator)).text
```

pages/login_page.py (inherits BasePage)

```
# pages/login_page.py
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class LoginPage(BasePage):
    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BTN = (By.ID, "login-button")
    ERROR_MSG = (By.CSS_SELECTOR, "h3[data-test='error']")

    def load(self):
        self.driver.get("https://www.saucedemo.com")

    def login(self, username, password):
        self.type(self.USERNAME, username)
        self.type(self.PASSWORD, password)
        self.click(self.LOGIN_BTN)

    def get_error(self):
        return self.get_text(self.ERROR_MSG)
```

Utility to Read CSV Data

utils/data_reader.py

```
# utils/data_reader.py
import csv

def read_csv(path):
    with open(path) as f:
        return list(csv.DictReader(f))
```

Converting a Test Case to the Page Object Model Framework

test_login.py (POM)

```
# test_login.py (final POM-based test - zero locators, zero raw Selenium calls)
import pytest
from pages.login_page import LoginPage

def test_valid_login(driver):
    login_page = LoginPage(driver)
    login_page.load()
    login_page.login("standard_user", "secret_sauce")
    assert "inventory.html" in driver.current_url

def test_invalid_login(driver):
    login_page = LoginPage(driver)
    login_page.load()
    login_page.login("standard_user", "wrong_pass")
    assert "do not match" in login_page.get_error()
```

Taking Screenshots on Test Failure

conftest.py

```
# conftest.py
import pytest
from utils.screenshot import take_screenshot

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        driver = item.funcargs.get("driver")
        if driver:
            take_screenshot(driver, item.name)
```

Module 2 Summary: You learned both of Python's major test frameworks — unittest (built-in, xUnit-style) and PyTest (industry standard, fixture-driven, plugin-rich) — and how to structure real projects using the Page Object Model so locators, actions, and assertions each live in exactly one place. Module 3 builds on this by automating REST APIs and introducing Behavior-Driven Development (BDD).